Q1) Create a table called Orders with columns: order_id, user_id, order_amount, and app_name (e.g., 'Zomato', 'Swiggy', 'Flipkart'). Insert at least 10 sample records with different users and apps. Write an SQL query using the OVER() function to display each order's amount along with the total order amount for all orders.

```
SELECT
    order_id,
    user_id,
    order_amount,
    app_name,
    SUM(order_amount)
OVER() AS
total_order_amount
FROM Orders;
```

Q2) Using the Orders table, write an SQL query to show each user's order_id, order_amount, and the average order_amount for that user using the OVER(PARTITION BY user_id) clause.

Hint: Use AVG(order_amount) OVER(PARTITION BY user_id) to get the per-user average.

```
SELECT
    user_id,
    order_id,
    order_amount,
    AVG(order_amount)
OVER (
        PARTITION BY
user_id
    ) AS
average_order_amount
FROM Orders;
```

Q3) Suppose you have a table called Playlist with columns: song_id, user_id, and duration_sec. Write an SQL query to display each song's duration, and the total duration of songs added by each user using SUM(duration_sec) OVER(PARTITION BY user_id).

```
SELECT
    song_id,
    user_id,
    duration_sec,
    SUM(duration_sec)
OVER (
        PARTITION BY
user_id
    ) AS
total_duration_sec
FROM Playlist;
```

Q4) Given a table named MovieRatings with columns: rating_id, user_id, movie_name, and rating (1-5), write an SQL query to show each rating, the average rating per movie, and the difference between the user's rating and the movie's average rating using window functions.

Hint: Use AVG(rating) OVER(PARTITION BY movie_name) and subtract it from the user's rating.

```
SELECT
    rating_id,
    user_id,
    movie_name,
    rating,
    AVG(rating) OVER (
        PARTITION BY
movie_name
    ) AS
average_movie_rating,
    rating - AVG(rating)
OVER (
        PARTITION BY
movie_name
    ) AS
rating_difference
FROM MovieRatings;
```